fractal

Session 5:

Ship Your PoC

Certificate in AI-Assisted Development | Session 4 | ~2 Hours | Claude Code | Final Session

Session 3 Recap

What You Produced

DEBUGGED

- ✓ 5-step debugging loop applied to a broken project
- ✓ Root cause named before every fix
- ✓ Regression test written and passing
- ✓ Habit established: read the diff before accepting

REVIEWED

- ✓ Structured review run on your own PoC
- ✓ Issues classified: Critical, High, Medium, Low
- ✓ Critical and High issues fixed and re-verified
- ✓ Medium issues noted in README

COMMITTED & DOCUMENTED

- ✓ Reviewed branch created with a descriptive commit message
- ✓ HANDOFF.md generated, edited, and committed to Git

You have a working, reviewed, documented PoC. Today you find out what it would actually take to ship it.

fractal

01 PoC To Production

Why Most PoCs Never Ship

The problem is rarely the PoC itself. It is the gap between what a PoC proves and what production requires.

THE DEMO PROBLEM

- Built for one question, one dataset, one predictable input.
- Production has none of these.

THE COMPLEXITY PROBLEM

- Auth, error recovery, audit logging, monitoring, rate limiting — none of it was forgotten. It was deliberately left out.
- Production puts it all back.

THE EXPECTATION PROBLEM

- Stakeholder saw a working demo.
- Engineering sees 10–20% of what production needs.
- Both are right. This session bridges that gap.

A PoC that never ships is not a failure. A PoC whose production cost was never estimated is a missed conversation.

The Production Gap

What Claude Code builds vs. what production needs. Production requires everything you forgot to ask for.

WHAT YOUR PoC HAS	WHAT PRODUCTION NEEDS
Core logic that runs	Core logic that runs at scale, with concurrent users
Hardcoded credentials	Secrets management and credential rotation
CLI output	Logging, monitoring, and alerting
One tested path	Full error handling and graceful failure
A README	Architecture docs, API contracts, runbooks
Committed to Git	CI/CD pipeline and automated tests
One user, you	Authentication and authorisation
Works on your machine	Works on any machine, any time, any load

None of the right column is a criticism of the PoC. All of it is the honest cost of production.

Technical Gaps from AI Code

In general, AI-generated code has a predictable pattern of gaps. Knowing them lets you find them before engineering does.

No input validation	Hardcoded secrets	No retry logic	Brittle dependencies
• Built for the input you described, not the one you forgot • Every unvalidated input is a crash waiting to happen	• API keys and passwords written directly into the code • Fine in a PoC. A security incident in production.	• External API fails, the script crashes • Claude will not add retry or graceful failure unless asked	• Library versions from Claude's training data • May not be current, maintained, or org-approved

These are not mistakes. They are the natural output of code built for proof, not production. To fix this, we have 10x rule.

The 10x Rule

A practical framework for estimating the real engineering effort to productionise a PoC, and for communicating it honestly to stakeholders.

THE RULE

For every unit of effort it took to build the PoC, estimate 10 units to make it production-ready.

A PoC built in one session means roughly 10 sessions of engineering effort to ship it.

This is a heuristic, not a formula. The multiplier shrinks for simple scripts with no external dependencies, and grows for anything touching authentication, compliance, or scale.

WHY IT EXISTS

The 10x rule is not pessimism, it is calibration. The gap is structurally large because:

- Security, compliance, and observability were all out of scope by design
- AI-generated code assumes ideal conditions; production provides none
- Code that works once needs to work every time, for every user, when things go wrong

When a stakeholder asks, "how hard to build can it be?", the 10x rule is your answer.

fractal

Production Gap Audit

HANDS-ON LAB

Apply the gap checklist to your own PoC. Name every gap, estimate effort, classify each as blocking or deferrable.

1 **Run the gap audit prompt**

```
claude "Review this PoC against a production checklist: input
validation, secrets management, error handling, retry logic,
logging, tests, auth, docs. Flag each as present, missing, or partial."
```

2 **Classify each gap**

Blocking (must close before production) · Deferrable (can ship with a known limitation) · Not applicable

3 **Apply the 10x rule**

Estimate the engineering effort to close every blocking gap. Name the total honestly.

4 **Answer the four greenlight questions**

One sentence each. Be honest.

5 **Identify your most surprising gap**

Not the most obvious one. The one you genuinely did not expect.

✓ **Gap audit complete · Each gap classified · 10x estimate named · Greenlight questions answered · Surprising gap identified**

fractal

Thank You.

fractal

A strategic partner to the most admired Fortune 500® companies globally, we help power every human decision in the enterprise by bringing advanced analytics & AI, engineering and design.

www.fractal.ai